



SMART CONTRACT AUDIT REPORT

for

PawnFi Protocol



Prepared By: Xiaomi Huang

PeckShield
August 23, 2024

Document Properties

Client	PawnFi
Title	Smart Contract Audit Report
Target	PawnFi
Version	1.0.2
Author	Xuxian Jiang
Auditors	Colin Zhong, Jinzuo Shen, Xuxian Jiang
Reviewed by	Patrick Lou
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0.2	August 23, 2024	Xuxian Jiang	Post Final #2
1.0.1	September 27, 2023	Xuxian Jiang	Post Final #1
1.0	September 12, 2023	Xuxian Jiang	Final Release

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About PawnFi	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	8
2	Findings	10
2.1	Summary	10
2.2	Key Findings	11
3	Detailed Results	12
3.1	Improved Validation on Protocol Parameters	12
3.2	Redundant State/Code Removal	13
3.3	Potential Rounding Issue in CToken::_redeemFresh()	14
3.4	Immutable States If Only Set At Constructor()	16
3.5	Improved Gas Efficiency in FundingStrategy	17
3.6	Possible Sandwich/MEV For Reduced Return	18
3.7	Improved NFT Withdrawal Logic in ApeStaking	19
3.8	Strengthened NFT Owner Verification in ApeStaking	20
3.9	Strengthened onlyEOA Verification	21
4	Conclusion	22
	References	23

1 | Introduction

Given the opportunity to review the `PawnFi` design document and related smart contract source code, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of `PawnFi` can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About PawnFi

`PawnFi` is a leading provider of instant liquidity solutions for `NFTs`. By leveraging the `P-Token` mechanism and integrating multi-modal features, `PawnFi` is designed to unlock deep liquidity and tap into the potential of `NFTs` without requiring ownership or digital asset transfers. In contrast, existing platforms like `Blur/OpenSea` offer a semi-primary market, where tokens are traded among different whales and reach fewer buyers or users, resulting in a market ceiling. The `P-Token` mechanism provides a protective layer that safeguards `NFTs` against extreme circumstances, enabling them to circulate safely across the broader DeFi ecosystem. The basic information of `PawnFi` is as follows:

Table 1.1: Basic Information of PawnFi

Item	Description
Name	PawnFi
Website	https://pawnfi.com
Type	Ethereum Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	August 23, 2024

In the following, we show the Git repositories of reviewed files and the commit hash values used in this audit.

- <https://github.com/PawnFi/DAO.git> (0fdd271)
- <https://github.com/PawnFi/Lending.git> (c9005fa)
- <https://github.com/PawnFi/LiquidityBoosting.git> (213c690)
- <https://github.com/PawnFi/Marketplace.git> (eb8aa95)
- <https://github.com/PawnFi/NFTFactory.git> (8d22da8)
- <https://github.com/PawnFi/ApeStaking.git> (71e2594)
- <https://github.com/PawnFi/Tools.git> (97fbb57)

And here are the commit IDs after all fixes for the issues found in the audit have been checked in:

- <https://github.com/PawnFi/DAO.git> (77987c4)
- <https://github.com/PawnFi/Lending.git> (5a635ec)
- <https://github.com/PawnFi/LiquidityBoosting.git> (93c9c51)
- <https://github.com/PawnFi/Marketplace.git> (04a53c0)
- <https://github.com/PawnFi/NFTFactory.git> (e8d83bd)
- <https://github.com/PawnFi/ApeStaking.git> (57d2e47)
- <https://github.com/PawnFi/Tools.git> (8c04310, d68635c)

1.2 About PeckShield

PeckShield Inc. [13] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [12]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [11], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.



Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the `PawnFi` implementation. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	
Low	5	
Informational	3	
Total	9	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability, 5 low-severity vulnerabilities, and 3 informational recommendations.

Table 2.1: Key PawnFi Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Improved Validation on Protocol Parameters	Coding Practices	Resolved
PVE-002	Informational	Redundant Data/Code Removal	Coding Practices	Resolved
PVE-003	Medium	Potential Rounding Issue in CToken::_redeemFresh()	Numeric Errors	Resolved
PVE-004	Informational	Suggested immutable States if Initialized Only in Constructor	Coding Practices	Resolved
PVE-005	Informational	Improved Gas Efficiency in FundingStrategy	Coding Practices	Resolved
PVE-006	Low	Possible Sandwich/MEV For Reduced Return	Time And State	Confirmed
PVE-007	Low	Improved NFT Withdrawal Logic in ApeStaking	Business Logic	Resolved
PVE-008	Low	Strengthened NFT Owner Verification in ApeStaking	Business Logic	Resolved
PVE-009	Low	Strengthened onlyEOA Verification	Coding Practices	Confirmed

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

3 | Detailed Results

3.1 Improved Validation on Protocol Parameters

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Multiple Contracts
- Category: Coding Practices [7]
- CWE subcategory: CWE-1126 [1]

Description

DeFi protocols typically have a number of system-wide parameters that can be dynamically configured on demand. The `PawnFi` protocol is no exception. Specifically, if we examine the `FeeManager` contract, it has defined a number of protocol-wide risk parameters, such as `feeRate` and `markets`. In the following, we show the corresponding routines that allow for their changes.

```

129     function setFeeRate(uint256 newFeeRate) external onlyOwner {
130         emit FeeRateUpdate(feeRate, newFeeRate);
131         feeRate = newFeeRate;
132     }
133
134     /**
135      * @notice Set market address corresponding to the asset
136      * @param newAsset Asset address
137      * @param newMarket market address
138      */
139     function setMarket(address newAsset, address newMarket) external onlyOwner {
140         getMarket[newAsset] = newMarket;
141         emit SetMarket(newAsset, newMarket);
142     }

```

Listing 3.1: `FeeManager::setFeeRate()/setMarket()`

These parameters define various aspects of the protocol operation and maintenance and need to exercise extra care when configuring or updating them. Our analysis shows the update logic on these parameters can be improved by applying more rigorous sanity checks. Based on the current

implementation, certain corner cases may lead to an undesirable consequence. For example, an unlikely mis-configuration of `feeRate` may charge unreasonably high lending market fee distribution rate, hence incurring cost or hurting the adoption of the protocol.

Recommendation Validate any changes regarding these system-wide parameters to ensure they fall in an appropriate range.

Status The issue has been fixed by this commit: 005b695.

3.2 Redundant State/Code Removal

- ID: PVE-002
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `ITokenStaking`
- Category: Coding Practices [7]
- CWE subcategory: CWE-563 [4]

Description

The PawnFi protocol makes good use of a number of reference contracts, such as `IERC20Upgradeable`, `SafeERC20Upgradeable`, `ReentrancyGuardUpgradeable`, and `OwnableUpgradeable`, to facilitate its code implementation and organization. For example, the `ITokenStaking` smart contract has so far imported at least five reference contracts. However, we observe the inclusion of certain unused code or the presence of unnecessary redundancies that can be safely removed.

For example, if we examine closely the `ITokenStaking` contract, there is a `_getRewardData()` routine to query updated reward data for the given token. Our analysis shows the internal `if`-condition (line 217) is redundant and can be safely removed.

```

202     function _getRewardData(address token) internal view returns (uint256
        accRewardPerShare, uint256 rewardsPerSecond) {
203         PoolInfo storage pool = poolInfo[token];
204         uint256 lpSupply = pool.adjustedSupply;
205         uint256 start = startTime;
206         uint256 currentWeek = (block.timestamp - start) / WEEK;
207
208         if (lpSupply == 0) {
209             return (0, incentiveVoting.getRewardsPerSecond(token, currentWeek));
210         }
211
212         uint256 lastRewardTime = pool.lastRewardTime;
213         uint256 rewardWeek = (lastRewardTime - start) / WEEK;
214         rewardsPerSecond = pool.rewardsPerSecond;
215         uint256 reward;
216         uint256 duration;

```

```

217         if (rewardWeek < currentWeek) {
218             while (rewardWeek < currentWeek) {
219                 uint256 nextRewardTime = (rewardWeek + 1) * WEEK + start;
220                 duration = nextRewardTime - lastRewardTime;
221                 reward = reward + duration * rewardsPerSecond;
222                 rewardWeek += 1;
223                 rewardsPerSecond = incentiveVoting.getRewardsPerSecond(token, rewardWeek
                );
224                 lastRewardTime = nextRewardTime;
225             }
226         }
227
228         duration = block.timestamp - lastRewardTime;
229         reward = reward + duration * rewardsPerSecond;
230         return (reward * 1e12 / lpSupply, rewardsPerSecond);
231     }

```

Listing 3.2: ITokenStaking::_getRewardData()

Recommendation Consider the removal of the redundant state (or code) with a simplified, consistent implementation.

Status The issue has been fixed by this commit: 005b695.

3.3 Potential Rounding Issue in CToken::_redeemFresh()

- ID: PVE-003
- Severity: Medium
- Likelihood: Low
- Impact: High
- Target: CToken
- Category: Numeric Errors [10]
- CWE subcategory: CWE-190 [2]

Description

The Lending component in PawnFi is in essence an over-collateralized lending pool that has the lending functionality and supports a number of normal lending functionalities for supplying and borrowing users, i.e., `mint()/redeem()` and `borrow()/repay()`. While reviewing the redeem logic, we notice the current implementation has a precision issue that has been reflected in a recent *HundredFinance* hack.

To elaborate, we show below the related `redeemFresh()` routine. As the name indicates, this routine is designed to redeems `CTokens` in exchange for the underlying asset. When the user indicates the underlying asset amount (via `redeemUnderlying()`), the respective `redeemTokens` is computed as `redeemTokens = div_(redeemAmountIn, exchangeRate)` (line 656). Unfortunately, the current approach may unintentionally introduce a precision issue by computing the `redeemTokens` amount against the protocol. Specifically, the resulting flooring-based division introduces a precision loss, which may

be just a small number but plays a critical role when certain boundary conditions are met – as demonstrated in the recent `HundredFinance` hack: <https://blog.hundred.finance/15-04-23-hundred-finance-hack-post-mortem-d895b618cf33>.

```

480     function redeemFresh(address payable redeemer, uint redeemTokensIn, uint
         redeemAmountIn) internal {
481         require(redeemTokensIn == 0 || redeemAmountIn == 0, "one of redeemTokensIn or
            redeemAmountIn must be zero");

483         /* exchangeRate = invoke Exchange Rate Stored() */
484         Exp memory exchangeRate = Exp({mantissa: exchangeRateStoredInternal() });

486         uint redeemTokens;
487         uint redeemAmount;
488         /* If redeemTokensIn > 0: */
489         if (redeemTokensIn > 0) {
490             /*
491              * We calculate the exchange rate and the amount of underlying to be
              * redeemed:
492              *   redeemTokens = redeemTokensIn
493              *   redeemAmount = redeemTokensIn x exchangeRateCurrent
494              */
495             redeemTokens = redeemTokensIn;
496             redeemAmount = mul_ScalarTruncate(exchangeRate, redeemTokensIn);
497         } else {
498             /*
499              * We get the current exchange rate and calculate the amount to be redeemed:
500              *   redeemTokens = redeemAmountIn / exchangeRate
501              *   redeemAmount = redeemAmountIn
502              */
503             redeemTokens = div_(redeemAmountIn, exchangeRate);
504             redeemAmount = redeemAmountIn;
505         }

507         /* Fail if redeem not allowed */
508         uint allowed = comptroller.redeemAllowed(address(this), redeemer, redeemTokens);
509         if (allowed != 0) {
510             revert RedeemComptrollerRejection(allowed);
511         }
512         ...
513     }

```

Listing 3.3: `CToken::redeemFresh()`

Recommendation Properly revise the above routine to ensure the precision loss needs to be computed in favor of the protocol, instead of the user. In particular, we need to ensure that markets are never empty by minting small `CToken` balances at the time of market creation so that we can prevent the rounding error being used maliciously. A deposit as small as 1 wei is sufficient.

Status The issue has been resolved as the team ensures the supported lending markets will not

be empty.

3.4 Immutable States If Only Set At Constructor()

- ID: PVE-004
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: RewardDistributor
- Category: Coding Practices [7]
- CWE subcategory: CWE-561 [3]

Description

Since version 0.6.5, [Solidity](#) introduces the feature of declaring a state as `immutable`. An `immutable` state variable can only be assigned during contract creation, but will remain constant throughout the life-time of a deployed contract. The main benefit of declaring a state as `immutable` is that reading the state is significantly cheaper than reading from regular storage, since it is not stored in storage anymore. Instead, an `immutable` state will be directly inserted into the runtime code.

This feature is introduced based on the observation that the reading and writing of storage-based contract states are gas-expensive. Therefore, it is always preferred if we can reduce, if not eliminate, storage reading and writing as much as possible. Those state variables that are written only once are candidates of `immutable` states under the condition that each fits the pattern, i.e., “a constant, once assigned in the constructor, is read-only during the subsequent operation.”

While examining all the state variables defined in the `RewardDistributor` contract, we observe there are several variables that need not to be updated dynamically. Examples include `reclaimPeriod` and `token`. They can be declared as `immutable` for gas efficiency.

```

10 contract RewardDistributor is Ownable {
11     using SafeERC20 for IERC20;
12     uint256 public reclaimPeriod;
13     address public token;
14     bytes32 public merkleRoot;
15     mapping(bytes32 => bool) public claimed;
16     ...
17 }

```

Listing 3.4: Key Storage States Defined in `RewardDistributor`

Recommendation Revisit the state variable definition and make good use of `immutable`/`constant` states.

Status The issue has been addressed by the following commit: `5c0e369`.

3.5 Improved Gas Efficiency in FundingStrategy

- ID: PVE-005
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: FundingStrategy
- Category: Coding Practices [7]
- CWE subcategory: CWE-561 [3]

Description

The PawnFi protocol has a built-in `FundingStrategy` contract to facilitate the raising of both `PToken` and `ETH` to provide liquidity. While reviewing the current logic, we notice the implementation to withdraw funds from lending markets can be improved.

In the following, we show the implementation of the related `_withdrawLendPools()` routine. It has a straightforward logic in withdrawing all funds from lending markets. And it comes to our attention that the withdrawal amount is already initialized in `itoken0` and `itoken1` (lines 545-546). However, the routine still reads from the storage (lines 548 and 557). With that, we suggest to simply reuse the variables `itoken0` and `itoken1`.

```

541     function _withdrawLendPools(uint256 rId, address token0, address token1) private {
542         InvestmentInfo storage investmentInfo = _investmentInfoMap[rId];
543         uint256 amount0;
544         uint256 amount1;
545         uint256 itoken0 = investmentInfo.itokens[token0];
546         uint256 itoken1 = investmentInfo.itokens[token1];
547         if(itoken0 > 0) {
548             uint256 income0 = _withdrawLendPool(lendPools[token0], token0,
                    investmentInfo.itokens[token0]);
549             if(income0 >= investmentInfo.remainingAmounts[token0]) {
550                 investmentInfo.bonus[token0] = income0 - investmentInfo.remainingAmounts
                    [token0];
551             }
552             amount0 = income0;
553         } else {
554             amount0 = investmentInfo.remainingAmounts[token0];
555         }
556         if(itoken1 > 0) {
557             uint256 income1 = _withdrawLendPool(lendPools[token1], token1,
                    investmentInfo.itokens[token1]);
558             if(income1 >= investmentInfo.remainingAmounts[token1]) {
559                 investmentInfo.bonus[token1] = income1 - investmentInfo.remainingAmounts
                    [token1];
560             }
561             amount1 = income1;
562         } else {
563             amount1 = investmentInfo.remainingAmounts[token1];
564         }

```

```

565     IERC20Upgradeable(token0).safeTransfer(liquidityBoosting, amount0);
566     IERC20Upgradeable(token1).safeTransfer(liquidityBoosting, amount1);
567 }

```

Listing 3.5: FundingStrategy::_withdrawLendPools()

Recommendation Improve the above logic to avoid repeatedly reading from the same storage.

Status The issue has been fixed by this commit: [a3385cf](#).

3.6 Possible Sandwich/MEV For Reduced Return

- ID: PVE-006
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: FundingStrategy
- Category: Time and State [9]
- CWE subcategory: CWE-682 [5]

Description

As mentioned in Section 3.5, the FundingStrategy contract facilitates the raising of both PToken and ETH to provide liquidity. In the process of examining the strategy logic, we notice the exit implementation needs to be aware of possible MEV risks.

```

501     function exitedStrategy(uint256 rId) public {
502         require(msg.sender == liquidityBoosting, "caller isn't liquidity offer");
503         InvestmentInfo storage investmentInfo = _investmentInfoMap[rId];
504         address token0 = IUniswapV3Pool(investmentInfo.pool).token0();
505         address token1 = IUniswapV3Pool(investmentInfo.pool).token1();
506         require(investmentInfo.returnAmounts[token0] == 0 && investmentInfo.
                    returnAmounts[token1] == 0, "exited strategy");
507         _removeLiquidity(rId, token0, token1);
508         _withdrawLendPools(rId, token0, token1);
509     }
510
511     /**
512     * @notice Remove liquidity
513     * @param rId Event ID
514     * @param token0 token0 address
515     * @param token1 token1 address
516     */
517     function _removeLiquidity(uint256 rId, address token0, address token1) private {
518         _collectFees(rId, token0, token1, true);
519
520         InvestmentInfo storage investmentInfo = _investmentInfoMap[rId];
521         INonfungiblePositionManager.DecreaseLiquidityParams memory params =
                    INonfungiblePositionManager.DecreaseLiquidityParams({

```

```

522         tokenId: investmentInfo.tokenId,
523         liquidity: investmentInfo.liquidity,
524         amount0Min: 0,
525         amount1Min: 0,
526         deadline: block.timestamp
527     });
528     (uint256 amount0, uint256 amount1) = INonfungiblePositionManager(
529         nonfungiblePositionManager).decreaseLiquidity(params);
530     _collectFees(rld, token0, token1, false);
531     investmentInfo.returnAmounts[token0] = amount0;
532     investmentInfo.returnAmounts[token1] = amount1;
533 }

```

Listing 3.6: FundingStrategy:: exitedStrategy ()

To elaborate, we show above the related `exitedStrategy()` routine. We notice the conversion is routed to `UniswapV3's nonfungiblePositionManager` in order to remove the liquidity. And the removal operation does not specify any restriction on possible slippage and is therefore vulnerable to possible front-running attacks, resulting in a smaller gain for this round of liquidity withdrawal.

Recommendation Develop an effective mitigation (e.g., slippage control) to the above front-running attack to better protect the interests of farming users.

Status The issue has been confirmed.

3.7 Improved NFT Withdrawal Logic in ApeStaking

- ID: PVE-007
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: ApeStaking
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [6]

Description

In `PawnFi`, there is an `ApeStaking` contract that is designed to streamline the staking process, allowing users to effortlessly stake their `APE` coins and benefit from compounded rewards through automatic reinvestment. While examining its `NFT-withdrawing` logic, we notice the current implementation may be improved.

To elaborate, we show below the implementation of the related `_withdraw()` function. As the name indicates, this function is used to withdraw an `NFT`, either paired or not paired. When it is not paired, there is a need to ensure the given `nftAddr` is not `BAKC_ADDR`. In other words, the `if`-condition (line 669) can be improved as `if( !paired && nftAddr != BAKC_ADDR)`.

```

664     function _withdraw(address userAddr, address nftAsset, uint256 nftId, bool paired)
        internal {
665         NftInfo storage nftInfo = _nftInfo[nftAsset];
666         require(userAddr == nftInfo.depositor[nftId], "d");
667         require(nftInfo.staker[nftId] == address(0), "s");
668
669         if(!paired) {
670             (uint256 tokenId, bool isPaired) = IApeCoinStaking(apeCoinStaking).mainToBakc
                (nftInfo.poolId, nftId);
671             if(isPaired){
672                 require(_nftInfo[BAKC_ADDR].staker[tokenId] == address(0), "p");
673             }
674         }
675         _withdrawNftFromLending(userAddr, nftAsset, nftId);
676     }

```

Listing 3.7: ApeStaking::_withdraw()

Recommendation Revise the `if`-condition in the above `_withdraw()` routine.

Status The issue has been fixed by this commit: [e00bc7d](#).

3.8 Strengthened NFT Owner Verification in ApeStaking

- ID: PVE-008
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: ApeStaking
- Category: Business Logic [8]
- CWE subcategory: CWE-841 [6]

Description

As mentioned in Section 3.7, the `ApeStaking` contract is designed to streamline the staking process and allow users to effortlessly stake their `APE` coins and benefit from compounded rewards through automatic reinvestment. Often times, there is a need to validate the given `NFT` is owned by a specific user. Our analysis shows the helper routine for ownership verification can be improved.

To elaborate, we show below the implementation of the ownership verification routine, i.e., `_validOwner()` function. It firstly checks the internal `_nftInfo` array for ownership. If the ownership is not recorded there, it then checks the `NftGateway` contract. It comes to our attention that the `NftGateway` to query needs to be validated. In particular, it may be `NftGateway` or `NftSale`.

```

378     function _validOwner(address userAddr, address ptokenStaking, address nftAsset,
        uint256 nftId) internal view returns (bool) {
379         address holder = _nftInfo[nftAsset].depositor[nftId];
380         if(holder == address(0)) {

```

```

381         address nftOwner = IPTokenApeStaking(ptokenStaking).getNftOwner(nftId);
382         holder = INftGateway(nftOwner).nftOwner(userAddr, nftAsset, nftId);
383     }
384     return holder == userAddr;
385 }

```

Listing 3.8: ApeStaking::_validOwner()

Recommendation Strengthen the ownership verification in the above `_validOwner()` routine.

Status The issue has been fixed by this commit: 08a6c1a.

3.9 Strengthened onlyEOA Verification

- ID: PVE-009
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Multiple Contracts
- Category: Coding Practices [7]
- CWE subcategory: CWE-1126 [1]

Description

The PawnFi protocol is designed to restrict certain functionalities to EOA accounts. Naturally, it has provided a EOA-validating modifier, i.e., `onlyEOA`. With the introduction and adoption of abstract account (<https://ethereum.org/en/roadmap/account-abstraction/>), the modifier logic may need to be improved.

```

357     modifier onlyEOA() {
358         require(tx.origin == msg.sender, "Only EOA");
359         _;
360     }

```

Listing 3.9: SAFE::onlyEOA()

To elaborate, we show above an example `onlyEOA` implementation. Its logic may be improved by further validating its code size needs to be zero, i.e., `require(tx.origin == msg.sender && address(msg.sender).code.length==0)`.

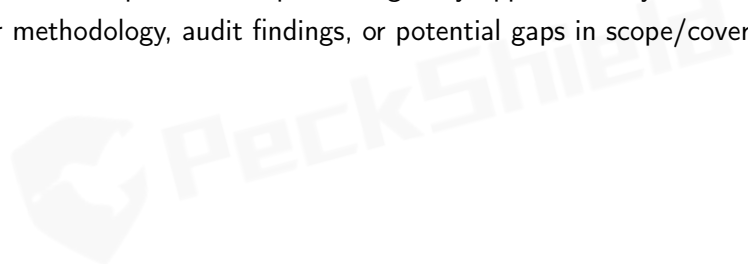
Recommendation Improve the above `onlyEOA()` modifier by further validating its code size.

Status The issue has been resolved by following the above recommendation.

4 | Conclusion

In this audit, we have analyzed the `PawnFi` design and implementation. The protocol is a leading provider of instant liquidity solutions for `NFTs`. By leveraging the `P-Token` mechanism and integrating multi-modal features, `PawnFi` is designed to unlock deep liquidity and tap into the potential of `NFTs` without requiring ownership or digital asset transfers. In contrast, existing platforms like `Blur/OpenSea` offer a semi-primary market, where tokens are traded among different whales and reach fewer buyers or users, resulting in a market ceiling. The `P-Token` mechanism provides a protective layer that safeguards `NFTs` against extreme circumstances, enabling them to circulate safely across the broader DeFi ecosystem. The current code base is clearly organized and those identified issues are promptly confirmed and resolved.

Meanwhile, we need to emphasize that smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. <https://cwe.mitre.org/data/definitions/1126.html>.
- [2] MITRE. CWE-190: Integer Overflow or Wraparound. <https://cwe.mitre.org/data/definitions/190.html>.
- [3] MITRE. CWE-561: Dead Code. <https://cwe.mitre.org/data/definitions/561.html>.
- [4] MITRE. CWE-563: Assignment to Variable without Use. <https://cwe.mitre.org/data/definitions/563.html>.
- [5] MITRE. CWE-682: Incorrect Calculation. <https://cwe.mitre.org/data/definitions/682.html>.
- [6] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. <https://cwe.mitre.org/data/definitions/841.html>.
- [7] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [8] MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- [9] MITRE. CWE CATEGORY: Error Conditions, Return Values, Status Codes. <https://cwe.mitre.org/data/definitions/389.html>.
- [10] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.

- [11] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [12] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.
- [13] PeckShield. PeckShield Inc. <https://www.peckshield.com>.

